EXPLORING DEEP LEARNING METHODS FOR FINE-TUNING PRETRAINED MODELS ON MULTI-RETINAL DISEASE DETECTION DATASET

Matt Stirling

University of Oulu
Faculty of Information Technology and Electrical Engineering
https://github.com/MattThePerson/DeepLearning2025_FinalProject

ABSTRACT

Transfer learning is often necessary in deep learning due to limited, good quality data and high training costs. This work evaluates pretrained ResNet18, EfficientNet-B0, and Swin Transformer models on a small, imbalanced subset of the ODIR retinal disease dataset, exploring various deep learning techniques. We find that full fine-tuning is preferrable to partial, attention mechanisms can be a good way to improve model performance, and that loss functions that address class imbalance must be applied carefully.

***Index Terms*—** transfer learning, fine-tuning, multi-label dataset, image classification, disease detection, attention mechanism, retinal disease

1. INTRODUCTION

Training deep learning models from scratch is difficult and resource intensive. It requires access to large, high quality datasets, which, for narrow deep learning tasks, may not exist. It is therefore desirable to leverage models that have been pretrained on large, diverse datasets by fine-tuning them on more task-specific ones, which can be comparatively much smaller. This technique is called transfer learning.

In this report, we explore various deep learning methods for fine-tuning pretrained models on a subset of the ODIR medical imaging dataset [1]. It consists of retinal images and our subset of it contains three disease classes: **Diabetic Retinopathy** (DR), **Glaucoma** (G), and **Age-related Macular Degeneration** (AMD). In total, we only have 1450 images, of which 800 are used in training and 250 are in the *onsite* test set, meaning we don't have the labels for the samples. There are about three times as many samples for the DR class compared to the other two disease classes. As such, this dataset presents an ideal opportunity to explore various deep learning methods for transfer learning.

As for our backbones, we have selected two image classification models which both represent advances in machine learning techniques. The first is *ResNet18* (2015) which introduced the concept of a "residual learning framework" in deep neural networks [2]. The second is *EfficientNet-b0* (2019) which belongs to the family of models called *EfficientNets* which are created via "neural architecture search" which has been demonstrated to improve on previous CNNs [3].

Both models have pretrained weights that were trained on the well known ImageNet dataset [4], and they both classify 1000 classes by default. For the purposes of our transfer learning approach, we modify the shape of the *classifier* (which usually means the final MLP or *fully connected layer*) so that it has an output shape of three.

We explore various techniques to attempt to achieve the best F1 score possible. This includes partial vs. full fine-tuning, alternative loss functions, augmenting the models with an attention mechanism, and lastly exploring the more powerful Swin Transformer model.

2. METHODS

For all our fine-tuning attempts, we mainly used the Adam [5] optimizer. We also implemented a final learning rate multiplier, so the learning rate could be gradually decreased over the training epochs. This technique proved very useful over fixed learning rate. We also often utilized a weight decay of between 10^{-5} and 10^{-7}.

2.1. Fine-tuning

For larger models, fine-tuning all the parameters can be prohibitively resource internsive. One approach may be to freeze the models backbone and fine-tune only the classifier, which may have orders of magnitude fewer parameters. Motivated by this idea, we fine-tuned both our models in two different ways: 1) only fine-tuning the classifier (freezing parameters in the backbone) and 2), fine-tuning all the parameters. For the sake of comparison, we also computed performance metrics for the pretrained weights (with the altered *classifier* shape) without any fine-tuning.

2.2. Loss functions

Due to the imbalanced nature of our ODIR dataset, we explored the potential benefits of using loss functions designed to address class-imbalance.

Each of these loss functions takes as input logits, or the raw outputs of the model which need to be processed (eg. with Softmax) to get the estimated probabilities.

2.2.1. Focal Loss

Focal loss is a criterion designed to improve model performance on hard-to-classify classes by down-weighting the loss computed for easy-to-classify samples [6]. It is defined as

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (1)$$

where p_t are the models estimated probabilities for the correct class and γ is a so-called *focusing* parameter. We settled on a γ of 0.5 as we found that larger values led to poor fine-tuning on our dataset.

2.2.2. Class-Balanced Loss

Another way to tweak the loss function to handle imbalanced classes is by applying *Class-Balanced Loss*. The idea is to weight each samples loss with a term that puts emphasis on the classes with fewer samples. It can be applied to any loss function, although we apply it to the *Binary Cross Entropy Loss*.

Formally speaking, we introduce an effective number term:

$$E_n = \frac{1 - \beta^n}{1 - \beta}, \quad (2)$$

where n is the number of samples in the class and β is a value between $[0, 1)$ which controls the strength of the reweighting effect. A β of 0 corresponds to no reweighting (multiplying by 1), whereas a β closer to 1 corresponds to a stronger class frequency weighting. This term is then inverted (so that we apply inverse class frequency weighting) and multiplied with the non-reduced loss output to get the *Class-Balanced Loss*:

$$\mathrm{CB}(\mathbf{p}, y) = \frac{1 - \beta}{1 - \beta^{n_i}} \mathcal{L}(\mathbf{p}, y) \quad (3)$$

where n_i is the number of samples in class i.

2.3. Attention mechanisms

One way to improve the performance of a computer vision model is by applying the concept of attention. The idea is to have the model determine the importance of different areas of activation and allow other areas to reference or learn from them, just like the brain might focus more on a specific area in its vision input, or in what its hearing.

In deep learning, there are four main categories of attention mechanism depending on what is being differentiated: spatial, channel, temporal, and branch attention [7]. For our purposes, we explored the use of a channel attention mechanism and a spatial attention mechanism.

2.3.1. Squeeze-and-Excitation

A Squeeze-and-Excitation (SE) block is an architectural unit which implements attention between channels by modelling interdependencies between them [8]. It uses two successive oprations: the **squeeze** operation, followed by the **excitation** operation. Loosely speaking, the **squeeze** step produces embeddings for each channels *feature responses* and the **excitation** step uses the embeddings to produce modulation weights for each channel, determining their 'importance'. These SE blocks can be stacked together to form *SE Networks*.

To test the potential improvements this technique offers, we injected a single SE block into the forward pass of the *EfficientNet* model between the backbone and the classifier. To determine the number of *squeeze channels*, we used a reducing factor of 16 on the input channels (1280) to get a channel count of 80.

2.3.2. Multihead Attention

The landmark computer science paper "Attention Is All You Need" (2017) [9] introduced a novel and revolutionary deep learning network architecture called the *Transformer*. One of the building blocked used to contruct this architecture was the Multi-Head attention (MHA) layer. This layer allows the model to jointly attend to information from different representation subspaces.

We experimented with augmenting both *EfficientNet* and *ResNet* models by injecting MHA layer at various placements in the forward pass. For EfficientNet we simply injected a single MHA layers between the backbone and the classifier. For ResNet, we experimented with injecting one or more MHA layers between the layers 2, 3 and 4 of the backbone. The motivation for this was to see how the feature sizes ($W \times H$) of the layers would affect the attention mechanism (the layers in question have output shapes of 32x32, 16x16, and 8x8 respectively). We also settled on 8 parallel attention layers for the MHA layer, which was also used in the paper [9, Sec 3.2.2].

2.4. More powerful backbone

Lastly, we explored more powerful models and settled on the *Swin Transformer* (2021) [10]. It is a so-called *vision Transformer* model, and as the name suggests it implements the transformer architecture introduced in 2017 [9] into a computer vision model. The Swin Transformer can be used as a "general-purpose backbone" in tasks such as image classification or object detection.

For computation purposes, we chose the smallest version of this model, Swin Tiny.

3. RESULTS

3.1. Fine-tuning

The results for fine-tuning the two backbones are shown in Table 1. Full fine-tuning outperformed classifier only fine-tuning substantially. Also, EfficientNet generally outperformed ResNet, although ResNet did have a better F1 score for AMD disease class.

Table 1: Comparing fine-tuning type. Comparing *EfficientNet* and *ResNet18* in 3 difference fine-tuning configurations. Metrics computed on offsite test set. Best column results in **bold**.

Average Metrics

Model	Accuracy	Precision	Recall	F1-score
Eff-Base	0.657275	0.654577	0.654028	0.637823
Res-Base	0.605853	0.640856	0.521327	0.558732
Eff-ClsFT	0.797962	0.763019	0.734597	0.741971
Res-ClsFT	0.764976	0.746570	0.668246	0.685783
Eff-FullFT	**0.870237**	**0.865803**	**0.829384**	**0.841722**
Res-FullFT	0.859313	0.841929	0.819905	0.825802

Disease specific F1-scores for Full FT

Model	DR	Glaucoma	AMD	*Average*
Eff-FullFT	**0.892734**	**0.813953**	0.578947	*0.841722*
Res-FullFT	0.889655	0.729412	**0.634146**	*0.825802*

Key: *Base* = no fine-tuning, *ClsFT* = classifier fine-tuned, *FullFT* = full model fine-tuned

3.2. Loss function

Our attempts to address class imbalance with a different loss function were not altogether successful (Table 2). BCE generally outperformed Focal Loss (FL) and Class-Balanced (CB) loss. For some diseases, CB loss proved better (eg. AMD F1 and accuracy or DR precision), but for the average metrics were all in favour of BCE loss.

3.3. Attention mechanisms

Table 3 compares our two attention mechanism augmented models with our best performing previous model. We were unable to achieve higher F1-score for the offsite test set using an attention mechanism, although MHA resulted in the highest Accuracy and Recall. Interestingly, the SE model had the highest F1-score computed on the onsite test set, and this was in fact the highest F1-score of any fine-tuning attempt for the onsite test set (Table 6).

3.4. More powerful backbone

Table 4 compares the metrics for all three main backbones. As expected, the more powerful *Swin Transformer* (tiny) backbone performed best in average F1-score, and in fact it achieved the highest overall offsite test set F1-score. It also achieved the second highest onsite test F1-score (Table 6).

Table 2: Comparing effect of loss function on disease specific metrics. Using *EfficientNet* backbone. *BCE* corresponds to results for *Eff-FullFT* model from Table 1.

F1-score

Loss	DR	Glaucoma	AMD	*Average*
BCE	**0.892734**	**0.813953**	0.578947	**0.841722**
FL	0.843636	0.666667	0.553191	0.772256
CB	0.888112	0.738095	**0.634146**	0.826794

Accuracy

Loss	DR	Glaucoma	AMD	*Average*
BCE	**0.845000**	**0.920000**	0.920000	**0.870237**
FL	0.785000	0.845000	0.895000	0.810403
CB	0.840000	0.890000	0.**925000**	0.860474

Precision

Loss	DR	Glaucoma	AMD	*Average*
BCE	0.865772	**0.945946**	**0.687500**	**0.865803**
FL	0.859259	0.704545	0.520000	0.787957
CB	**0.869863**	0.885714	0.684211	0.854187

Key: *BCE* = Binary Cross-Entropy Loss, *FL* = Focal Loss, *CB* = Class-Balanced Loss

Table 3: Comparing attention mechanisms.

Average metrics

Mechanism	Accuracy	Precision	Recall	F1-score
None (EffNet)*	0.870237	**0.865803**	0.829384	**0.841722**
SE (EffNet)	0.856682	0.853387	0.815166	0.833390
MHA (ResNet)	**0.871943**	0.853045	**0.838863**	0.840947

* Best performing model without an attention mechanism (*EffNet-FullFT)*.

However, Swin was beaten in accuracy and recall by EfficientNet. Interestingly, despite having the highest *average* F1-score, Swin did not have the best disease specific F1-score for any of the three classes.

Table 4: Comparing more powerful backbone to *EfficientNet* and *ResNet*. Best metrics shown in **bold** and worst in *italic*.

Average metrics

Backbone	Accuracy	Precision	Recall	F1-score
EffNet	**0.870237**	**0.865803**	0.829384	0.841722
ResNet	*0.859313*	*0.841929*	*0.819905*	*0.825802*
Swin	0.869668	0.857718	**0.834123**	**0.844040**

Disease specific F1-scores

Backbone	DR	Glaucoma	AMD
EffNet	**0.892734**	**0.813953**	*0.578947*
ResNet	*0.889655*	*0.729412*	**0.634146**
Swin	0.890511	0.808511	0.627451

3.5. Comparison of best models and Onsite test set metrics

The onsite test set F1-scores for all fine-tuned models can be found in Table 6. Most tasks resulted in F1 scores that were comparable or better than the reference results.

Table 5 shows some of the average performance metrics of our top 5 best performing models (according to the onsite test set F1-score). It compares both onsite and offsite F1-scores, which are at times wildly out of sync.

Table 5: Best models full metrics comparison. Includes comparison for offsite and onsite F1-scores.

Top 5 best models

Model	Acc	Prec	Rec	Offsite F1	Onsite F1
EffNet	0.87023	**0.86580**	0.82938	0.84172	0.8134
CB-loss	0.86047	0.85418	*0.81042*	*0.82679*	0.8016
SE-EffNet	*0.85668*	0.85338	0.81516	0.83339	**0.8202**
MHA-Res	**0.87194**	*0.85304*	**0.83886**	0.84094	*0.7959*
Swin	0.86966	0.85771	0.83412	**0.84404**	0.8185

Table 6: Onsite test set results. CMP indicated whether F1-score is within reference error or greater. Top two results shown in **bold**. *(Hopefully this is helpful for the grading process.)*

Onsite Test Set Results

Task	Desc.	BCK	Ref. F1	F1	Diff %	CMP
task1-2	Par.	Eff	73.5 ± 0.6	69.52	-5.41%	False
task1-2	Par.	Res	61.4 ± 0.3	57.86	-5.77%	False
task1-3	Full	Eff	80.4 ± 0.5	81.35	+1.18%	**True**
task1-3	Full	Res	78.8 ± 0.8	78.20	-0.76%	**True**
task2-1	FL	Eff	80.4 ± 0.5	77.73	-3.33%	False
task2-2	CB	Eff	80.4 ± 0.5	80.16	-0.29%	**True**
task3-1	SE	Eff	80.4 ± 0.5	**82.03**	**+2.02%**	**True**
task3-2	MHA	Res	78.8 ± 0.8	79.59	+1.01%	**True**
task4		Swin	80.4 ± 0.5*	**81.85**	**+1.80%**	**True**

*Reference results for EfficientNet

4. DISCUSSION

Our journey in exploring deep learning techniques was at times a bit rocky. For one thing, during training, the validation loss and validation F1 scores would have little correlation. A higher learning rate would often get better F1 scores, but with an early dip in val loss, whereas a lower learning rate would result in a smoother val loss decrease, but never achieving as good F1, even with higher training epoch (250+). For this reason, we saved the best checkpoint both when validation was at its lowest *and* when validation F1 was at its highest. In fact, most of our good results came from best val F1 checkpoints, even though the val loss was much worse.

For MHA, it's interesting to note that although EfficientNet was generally better than ResNet, we achieved better results augmenting ResNet than we did EfficientNet. This is possibly due to the forward method being more explicit, allowing for more experimentation. Our final configuration was two MHA layers between backbone layers 3 and 4.

5. CONCLUSION

We believe that we have adequately explored the various deep learning techniques. We conclude that, in general, for smaller models full-fine tuning is preferrably to partial, classifier-only fine-tuning. Augmenting a model by adding an attention mechanism can be a great way to improve performance, but it has to be done carefully and involves a lot of experimentation. Implementing a loss function such as focal loss can be technically challenging, and loss criteria that try to fix class-imbalance issues must be applied carefully. Lastly, transformer based vision models seem to outperform regular CNNs.

References

[1] Ning Li, Tao Li, Chunyu Hu, Kai Wang, and Hong Kang, "A benchmark of ocular disease intelligent recognition: One shot for multi-disease detection," 2021, https://arxiv.org/abs/2102.07978.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," 2015, https://arxiv.org/abs/1512.03385.

[3] Mingxing Tan and Quoc V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020, https://arxiv.org/abs/1905.11946.

[4] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei, "Imagenet large scale visual recognition challenge," 2015.

[5] Diederik P. Kingma and Jimmy Ba, "Adam: A method for stochastic optimization," 2017, https://arxiv.org/abs/1412.6980.

[6] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár, "Focal loss for dense object detection," 2018, https://arxiv.org/abs/1708.02002.

[7] Meng-Hao Guo, Tian-Xing Xu, Jiang-Jiang Liu, Zheng-Ning Liu, Peng-Tao Jiang, Tai-Jiang Mu, Song-Hai Zhang, Ralph R. Martin, Ming-Ming Cheng, and Shi-Min Hu, "Attention mechanisms in computer vision: A survey," *Computational Visual Media*, vol. 8, no. 3, pp. 331–368, Sept. 2022, http://dx.doi.org/10.1007/s41095-022-0271-y.

[8] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu, "Squeeze-and-excitation networks," 2019, https://arxiv.org/abs/1709.01507.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention is all you need," 2023, https://arxiv.org/abs/1706.03762.

[10] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," 2021, https://arxiv.org/abs/2103.14030.